Lab 3: Welcome to `Qzicl` !

version 2.1.1 - Last updated: 2026-10-20T11:34:00-04



`Qzicl` ("KWIZ-i-kul") is a web application that enables users to select an area of knowledge and a sub-topic of the area from a list, then take a multiple-choice quiz to test their knowledge of that topic. It can be used for entertainment or as a study system for learning.

The Lab 3 assignment is to deliver the **server** that manages all of the quiz data, interacts with multiple front ends to oconduct multiple choice quizzes, and keeps statistics. You do not need to implement the front end. However, you will need to verify the implementation of the server is correct by using one of the API testing schemes (e.g., `SuperTest` or `REST Client`).

Lab 3 is due in GitHub Friday Nov 6 @ 22:00 EDT

> [!NOTE] You are free to use the fine open source API testing tools Bruno or Insomnia, or the paid API testing tools Postman or SmartBear.
>
> Some warnings:
>
> 1. Your submission must include a `SuperTest` and/or `REST Client` files to fully test your server. In other words, you have to do the testing with these tools anyway, and that testing is sufficient. Using any other tools would only be for practice with no effect on your grade.
> 2. If you do use one of these other tools to help you complete your Lab 3 work, you must include all test definition files in your submission and identify any parts written by the tool. I should be able to run the tool and use your files to show that your results are correct.
> 3. I do **not** suggest that you **pay** for anything. The free tools we discussed in class are fully capable to do the job.

Web application client/server flow

1. Your **qzicl** server is up and running.
2. The user starts a client app, which requests a quiz session with the `qzicl` server.
3. The server responds by initiating a quiz session and sends the client app a welcome message along with a list of the available topics with general information about each. For example, the topics might be "math", "history", or "science".
4. The user selects one of the topics by sending another request to the server.
5. Upon receiving the topic selection, the server responds to the client app with a list of the available quizzes for that topic. For example, if the topic is "*math*" the available quizzes might be "*geometry*", "*calculus*", and "*general math*".
6. The user selects one of the available quizzes by sending another request to the server.
7. The server receives the quiz selection and responds with a "get ready" message that contains a `sessionID` the user will use for the remainder of the quiz.
8. When the user is ready to begin, they send a "Go" request to the server. The server begins sending the client the quiz questions, one by one, with the miltiple choice answers to choose from. All the questions are multiple choice.
9. As each question and answer choices arrive, the user selects an answer and sends their choice to the server.
10. Upon receiving that answer choice, the server responds with the next question.
11. Quizzes can be of any length. After receiving the user's answer to the last question of a quiz, the server returns the results and the quiz session ends. The results include:

- the quiz topic and quiz title (e.g., "Math" and "Geometry"),
- the user score as "# correct out of # (xxx%)", and
- the total elapsed time the user spent taking this quiz.

Each time a question is sent to the user, the server expects them to submit an answer within 30 seconds. If the user has not responded within that time, the server will terminate the quiz session, delete any recorded results, and answer any subsequent messages from the user for that `sessionID` with a "session expired" message and status code.

If the client app crashes, the server will follow the same procedure. After 30 seconds have passed since the current question was sent to the user, that `sessionID` is noted as being "expired" and any subsequent message using that `sessionID` will get the "session expired" message and status code.

The user may choose to leave the quiz before it is finished. If so, the client sends a special message to the server for this purpose. Upon receiving that message, the server will terminate the quiz, delete any recorded results, and send a response to the user's client to exit the session.

The server will log all user interactions and any server-specific runtime information to its log file. The server is not expected to remember a used or expired `sessionID` across runs. Each time your server starts, it has no history.

Architecture

Your assignment is to build the `Qzicl` **server**. While you are developing the server, you should use an API testing tool to verify that your server is working properly. No `Qzicl` client will be provided for the assignment. Real-world Full-Stack Developers will now always have a client to use for testing. That's why they always use automated testing tools as a sort of "mega Unit Test" of their server. That way, when a client that implements the same API specification does become available, the server side is ready.

For this assignment, your server will fully implement the provided Qzicl v2.1 API.

> **Important**
> *It is your responsibility to ensure that the server accurately and completely implements the published API Specification*. Be assured, I will test it thoroughly.

> **Warning**
>
> I change the API every semester. So if you have access to solutions to this Lab from a previous semester, it will very likely take you longer to repair it than if you just did the work (and learned stuff!) yourself.

It is extremely important that you thoroughly test your server *during development* using API testing tools such as these (all of which run on Windows, MacOS, and Linux). **DO NOT** wait to do "Big Bang Testing" when you're finished.

This parallel development model is how web application development works in industry. The server development team works independently from the client development team. In fact, there are often multiple organizations working on clients on different platforms. But they *all* do their work based on a common API Specification.

API testing tools we discussed in class

Tool	Useage	Free?	link
REST Client	VSCode extension: REST Client by Huachao Mao that allows you to send `HTTP` requests and view the response in Visual Studio Code directly.	Free	REST Client
SuperTest	NPM Package	YES	SuperTest
curl	CLI tool for sending `HTTP` requests & receiving responses (text based). It is one of the most widely used software tools in the world, with an estimated **over 20 billion installations**	YES	curl

Other popular API testing tools

Tool	Useage	Free?	link
HTTPie	Downloadable app; Friendly and easy, with very limited import/export.	YES	HTTPie
ThunderClient	VSCode extension	Very useful Free-Tier	ThunderClient
Bruno	Webapp, desktop app, CLI, or VS Code extension; Alternative to Postman.	Very useful Free-Tier	Bruno
ApiDog	Webapp or desktop app. Alternative to Postman.	Limited Free-Tier	ApiDog
Postman	An industry-standard API platform used by developers to build, test, design, document, and manage APIs. Large, complex, supports teams, etc.	PAID 	Postman

The `Qzicl` server API

The `Qzicl` system uses a RESTful API.

A "RESTful API" refers to a web service offered by a server that follows the REST style, where client requests are submitted via `HTTP` `GET`, `POST`, `PUT`, and `DELETE` methods. The server manages some resources and responds accordingly, while treating each request as independent and self-contained (stateless).

The server responds to the routes and requests as shown in the Qzicl V2.1 API Specification document. Response status codes from the server are specified in the document.

The server runs on TCP/IP port `46500`, which is where all user clients (and the instructor) expect it to be listening. If that port is not available, the server immediately issues an error message and exits.

> **Tip**
>
> Can you think why the Qzicl server (and all the others on the Internet) always run on a specific port?

Sample quiz file

The deliverables for this assignment includes sample quizzes. The quizzes can be stored at the server in several ways:

- as JavaScript objects defined in separate files and included
  (*the Pro way*, easy to change without rebuilding)
- as JavaScript strings in separate files and included
  (*the ok way*, still easy to change without rebuilding)
- as JavaScript strings hard-coded into your main `.js` or `.jsx` files
  ("*the hack way*", hard to maintain, messy)

Here is an example of how the content of one of those quiz files might be defined.

```
{
  "topic": "General EU Knowledge",
  "title": "EU Capitals Quiz",
  "quizMessage": "Test your general knowledge with this sample quiz!",
  [
    {
      "question": "What is the capital of France?",
      "options": [
        "Berlin",
        "Madrid",
        "Paris",
        "Rome"
      ],
      "answer": "Paris"
    },
    {
      "question": "What EU capital is west-northwest of Brussels?",
      "options": [
        "Amsterdam",
        "Madrid",
        "Paris",
        "London"
      ],
      "answer": "London"
    },
    {
      "question": "The largest EU capital by area is?",
      "options": [
        "Rome",
        "Berlin",
        "Amsterdam",
        "Madrid"
      ],
      "answer": "Berlin"
    }
  ]
}
```

Lab3 specific requirements

Produce a complete, runnable `Qzicl` Server that fully implements the provided Qzicl v2.1 API Specification using Express. The server can handle all the quiz interactions described in this Fall 2026 version of the assignment. Specifically, the server should:

- Use the most recent versions of all the software used, including software obtained from the `npm` registry.
- Handle the REST API requests documented in the Qzicl v2.1 API Specification from one or more instances (users) of a `Qzicl` client (or API testing tool).
- Include what is required to run all tests (including API tests) used to ensure the server is operating properly. A test coverage report should show 80% or higher. Grading will include generating a test coverage report to verify this.
- Write a log file that summarizes all requests received, all responses sent, and any server-specific runtime information (e.g., "Server started at 2026-10-19T12:58:00-04"). Recommendation: use the easy-to-use `LogLevel` package.
- Include a "Safe Global Error Handler". The server should preserve the front end user experience by catching **ALL** unhandled errors. y testing should not cause a server crash or hang-up. These errors should be noted in the console log and an `500 Internal Server Error` response sent to the front end. The console log entry should include a decription of the error along with stack information when available. See the lecture notes from Oct 16 for more details.
- Be able to handle quizzes for many users at the same time. Since the API is RESTful, it is *stateless*.
- Provide sample quizzes containing:
  - **Two or more** topic areas
  - **Two or more** quizzes in each topic area
  - **Three or more** multiple-choice questions in each quiz
  - **Three or more** possible answers for each question

    this means a grand total of at least

$$2 \text{ topics} \times 2 \text{ quizzes} \times 3 \text{ questions} \times 3 \text{ answers} = 36 \text{ questions}$$

-

> **Important**
>
> The instructor should be able to clone your repository onto a laptop, `cd` into the created directory, run `npm install`, and `npm run server` to run it.
>
> If any other steps are necessary, they must be **fully** documented in your `README.md`. If the instructor doesn't know how to run your code properly, the resulting grade will not accurately represent your work.

> **Warning**

> The server *must not* depend on any specific operating system type or version, or any specific hardware architecture or setup.

What to submit

All the files required to build, test, and run the server (including the sample quizzes). In addition, you must include:

- **`README.md`**
  Update the provided `README.md` in the repository to include:
  - Your name after `Lab3 - CS465 Fall 2026` in the `Author` heading. This line is provided in the starter code `README.md` file.
  - List any prerequisite software and versions that your solution was built with in the ***`Prerequisites`*** section. I should be able to start in an empty directory and, after installing these prerequisites, be able to build and run your system.
  - Explain how to build your solution in the **`Building this lab`** section.

    > **Important**
    > If you do not provide complete instructions in the `README.md` file for me to build your Lab 2 submission, I will be unable to grade it. 😠

  - Describe the API test tool(s) and data used in the **`Testing procedures`** section.
  - Describe how to run the server in the **`Running the server`** section, including any prerequisite system preparation.
  - Describe any challenges you faced and how you overcame them in the **`Challenges`** section.
  - Add complete attribution of all sources you used including documentation, source code, libraries, coding examples, etc., in the **`Full attribution of sources`** section.

    > **Caution**
    > Any code you use that was created by another entity (human or AI ✨) must be fully acknowledged. Be sure you understand it and can explain it to me if asked. If you cannot explain it, don't use it.

  - If you implemented any extra credit features, you must describe them fully in the **`Extra credit`** section.

    > **Warning**
    > If you add any extra credit features and don't tell me about them here, I won't know about them, resulting in no extra credit for your extra work. 😞

  - **Resilience**: No programming error messages are ever exposed to the user, and only to the server

console log if it is important. Similarly, nothing the user can do will crash your server.

- **Code Quality**: Write clean, well-organized, and commented code to make it easy to understand and maintain. Choose *either* camelCase 🐫 or snake_case 🐍 for variables and functions. Remember: if I can't understand it, I can't grade it. 😖

  The `git` commit messages you write are a key part of Code Quality in the industry. They should be meaningful and precise.

- **Submission**: Regularly do `git commit` and `git push` of your work to insure against loss. Use branches to avoid stepping on previously completed work before you're ready to merge them. When finished, run your last `git commit` and a `git push` to send your work back to the repository on GitHub Classroom. After the due date/time, I will grade the contents of the `main` branch of that repository.

- An example log file created by your server when running.
- `gitignore` that is appropriate for Express / Node.js development and excludes the `node_modules` directory!
- Your `package.json` file should include appropriate values for `author`, `license`, and `private`.
- `LICENSE` file including your name and the date.

Submitting your work

This assignment is managed through the `classroom50.org` classroom for this class.

- You began by accepting the assignment via the link in the Lab 3 description on BrightSpace. Once accepted, a repository was created for you to `git clone` onto your laptop.
- You should do all your work on this assignment in the repository created for you by `classroom50`. Everything needed to build, test, and run your server should be in this repository.
- The repository created for you will initially contain the files needed to get started. Do not erase any of these files, unless directed to do so by the assignment or the instructor.
- Your work is submitted using `git push` to the repository created for you by `classroom50.org`. Only the most recent `git push` will be graded.
  - Nothing is submitted via Brightspace.
- The instructor will clone the contents of all Lab 3 repositories for grading after 22:00 (10 PM) Eastern Standard Time the day the assignment is due.

Extra Credit (maximum of +5 points)

> **Note**
>
> Extra credit may only be earned if the base grade for the assignment is at least 85%.

Qzicl deployed and running on Render.com (+1)

(or some other free hosting site). Provide the URL in the `README.md` file.

Topic/question editor (+2)

A standalone Question/Answer editor to create the JSON file for a set of Q&A's. I would save the Q&A's in an appropriate JSON file. Of course, it would ensure that the resulting JSON file is valid and would work with your server.

Server usage report (+2)

This includes

- a count of the total number of quizzes that were started, timed out, exited early, or were completed since the server was last started.
- a report of scores on individual questions across all completed topic quizzes since the server was last started.
- NO information on specific users of the server is maintained, so such data would not be included.

Settings screen (+2 points)

- Identify four or more useful settings to control how the server runs.
    - For example, the ability to specify which quiz question & answer files to use.
- Settings are stored in a JSON object in a file named `qzicl.config`. The file is read by your server when it is started.
- The server will check the settings file for validity before proceeding.
- Provide full documentation on what the settings do and how to specify them.

Images in Q&A (+2 points)

- Extend the system to allow the questions and answers to include text *or images*.
- Would the API need to be changed?

References

- `Loglevel` package
- ThunderClient
- HTTPie
- curl
- Vitest
- OpenAPI v3
- Bruno